

Contributing to Rev80

Dev environment setup, project layout, and build/packaging instructions. For usage, installation, configuration, and the vibration/DSP technical reference, see [README.md](#).

Table of Contents

- [Development environment](#)
 - [Moving the checkout](#)
 - [Rendering docs to PDF](#)
 - [Project layout](#)
 - [Testing](#)
 - [Replacing the app icon](#)
 - [Building the Windows Installer](#)
-

Development environment

Linux / macOS recommended.

```
git clone <repo-url>/rev80.git
cd rev80

# Create and activate a virtual environment
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate

# Install in editable mode with dev tools
pip install -e ".[dev]"

# Enable the repo's git hooks (once per clone) – keeps doc/*.pdf in sync
# with README.md, CONTRIBUTING.md, doc/PROGRESS.md, doc/CHANGELOG.md on commit
git config core.hooksPath .githubhooks

# Run the test suite (no hardware required – uses SimulatedSensor)
pytest tests/

# Launch the app against the source tree
python -m rev80
```

Moving the checkout

Three things outside the repo are bound to its **absolute path** and break silently when the checkout moves. This has happened on every relocation so far, so run all of it in one pass:

```
cd <new-path>

# 1. Re-point the editable install. Without this, `import rev80` and both console
# scripts fail with ModuleNotFoundError while `pytest` keeps passing – tests
# resolve via pyproject's `pythonpath = ["src"]`, which never consults the install.
```

```

pyenv local <env> # or: source .venv/bin/activate
python -c "import sys; print(sys.prefix)" # confirm the intended env FIRST
pip install -e ".[dev]"

# 2. Re-enable git hooks. Use the relative path – an absolute `core.hooksPath`
#   silently disables every hook (including the blocking `ruff` gate) after a move.
git config core.hooksPath .githooks

# 3. Verify
rev80 --list-sensors # must not raise ModuleNotFoundError
git config --get core.hooksPath # must print: .githooks

```

`.python-version` and `.vscode/` are gitignored so each checkout owns its own dev environment; they are not carried by `git clone`. Prefer `mv` over a fresh clone when relocating, or recreate them by hand. Uninstall any stale distribution left over from the `vibechecker` era with `pip uninstall vibechecker` — it is a separate dist and `pip install -e .` will not remove it.

Nothing *inside* the repo hardcodes an absolute path, and the user-level state (`~/ .config/rev80/`, `~/ Documents/Rev80/data/`, the desktop entry) is path-independent.

Rendering docs to PDF

`doc/README.pdf`, `doc/CONTRIBUTING.pdf`, `doc/PROGRESS.pdf`, and `doc/CHANGELOG.pdf` are committed alongside their Markdown sources. The same `pre-commit` hook that runs `ruff` (`.githooks/pre-commit`, enabled by the `git config core.hooksPath .githooks` step above) also re-renders and stages the PDF for any of those four files that are part of a commit, via `scripts/render_md.sh`:

```

# Render one file manually, e.g. after editing without committing
./scripts/render_md.sh README.md # → doc/README.pdf
./scripts/render_md.sh doc/CHANGELOG.md # → doc/CHANGELOG.pdf

```

Requires `pandoc` and `weasyprint` on PATH. If either is missing, the hook prints a warning and skips rendering rather than blocking the commit — install them to keep the PDFs current:

```

# Debian/Ubuntu
sudo apt-get install pandoc
pip install weasyprint

```

Project layout

<code>src/rev80/</code>	Python package
<code>_paths.py</code>	Runtime-safe path resolution (dev vs frozen)
<code>_pico_loader.py</code>	Windows DLL search path setup for frozen builds
<code>logging.yaml</code>	Logging configuration (bundled with package)
<code>monitor/</code>	Monitor Mode package
<code>__init__.py</code>	
<code>session.py</code>	MonitorSession dataclass
<code>gate.py</code>	IntervalGate scheduler
<code>anomaly.py</code>	AnomalyHook protocol; Rms/Spectral/FixedThreshold hooks;
Composite	
<code>writer.py</code>	MonitorWriterThread (daemon, writes session.h5)

controller.py	MonitorController (is_recording; hook supplied at start)
assets/	Package data – bundled into wheels and frozen builds alike
fonts/	CommitMono Nerd Font (gitignored; fetched by scripts/
fetch_font.sh)	
icons/hicolor/	App icon PNG set (16..256px), for `rev80 --install-desktop-
entry`	
	on Linux – see "Replacing the app icon" below
desktop.py	Linux .desktop launcher entry + icon install/uninstall
assets/	App icon source (edited directly by scripts/make_icons.sh)
rev80.svg	Source artwork
rev80.ico	Multi-resolution icon used by installer and exe
drivers/	PicoScope DLLs (Windows build only, not committed)
installer/	Inno Setup script
tests/	pytest suite
test_monitor_controller.py	
test_monitor_gate.py	
test_monitor_index.py	
test_monitor_session_load.py	full write→load→browse integration tests
build/	
rev80.spec	PyInstaller build spec
collect_pico_dlls.py	Collects PicoScope DLLs into drivers/
scripts/	
build.sh	Full build pipeline (run from Git Bash)
make_icons.sh	Regenerates rev80.ico + the hicolor PNG set via Inkscape +
ImageMagick	
fetch_font.sh	Downloads the CommitMono Nerd Font into src/rev80/assets/fonts/
render_md.sh	Renders a Markdown file to PDF (pandoc + weasyprint) – see doc/
.githubhooks/	
pre-commit	ruff check (blocking; scope matches CI), and re-renders doc/
*.pdf	
	for any of README.md, CONTRIBUTING.md, doc/PROGRESS.md,
	doc/CHANGELOG.md staged in the commit
doc/	
README.pdf, CONTRIBUTING.pdf, PROGRESS.pdf, CHANGELOG.pdf	Rendered by the pre-commit
hook	
PROGRESS.md	Delivery history – client requirements mapped to commits
CHANGELOG.md	Keep-a-ChangeLog-format change log

Testing

Tests use `VibeSensor.simulated()` and synthetic data — no hardware required. The full suite runs in ~50 s. Tests that write `.h5` scratch files use a project-relative `./DEVDATA/` directory (hardcoded in the test files, independent of the app's own `~/Documents/Rev80/data/` save location) — safe to delete between runs.

```
# Run all tests (no hardware)
pytest tests/

# Skip hardware-dependent tests explicitly
pytest tests/ -k "not hardware and not siggen"

# Single file
pytest tests/test_monitor_session_load.py

# Single test
pytest tests/test_vibechecker.py::test_save
```

Test file	Coverage
<code>test_sample.py</code>	<code>AcquisitionSettings</code> , <code>VibeSample</code> , <code>ChannelResult</code>
<code>test_vibechecker.py</code>	<code>DataCollector</code> stream lifecycle, save/load, trend
<code>test_acquisition_settings.py</code>	Derived properties, setter validation
<code>test_scope_sensor.py</code>	Sensor calibration pipeline, mV → EU scaling
<code>test_picoscope.py</code>	<code>FindPicoScope</code> enumeration logic (mocked driver)
<code>test_config.py</code>	YAML persistence, defaults, atomic writes
<code>test_monitor_gate.py</code>	<code>IntervalGate</code> snap-to-grid, burst entry/exit
<code>test_monitor_anomaly.py</code>	<code>RmsThresholdHook</code> (warmup, streak, t=0 tracking), <code>SpectralThresholdHook</code> (EWMA, band masking), <code>FixedThresholdHook</code> (unit conversion, mV skip, modality mismatch), <code>CompositeAnomalyHook</code>
<code>test_monitor_controller.py</code>	<code>MonitorController</code> lifecycle, hook wiring, cooldown gating, HDF5 structure
<code>test_monitor_session_load.py</code>	Full write → load → browse integration: interval frames, burst frames, trend reconstruction, NaN regression guard

Hardware-specific tests skip automatically when no PicoScope is detected.

Replacing the app icon

Edit `assets/icons/rev80.svg` , then regenerate both the Windows `.ico` and the Linux `hicolor` PNG set from it — no changes to `build/rev80.spec` , `installer/rev80.iss` , or `desktop.py` are needed either way:

```
# From the repo root — requires inkscape and imagemagick
./scripts/make_icons.sh
```

This updates `src/rev80/assets/icons/hicolor/*/apps/rev80.png` (Linux desktop entry) in place — commit it. It currently writes `rev80.ico` to the repo root rather than `assets/icons/rev80.ico` ; move it there manually before committing so `build/rev80.spec` picks up the update.

Building the Windows Installer

Produces a self-contained one-directory executable and a standalone installer (`.exe`) via PyInstaller and Inno Setup. **The build must run on a 64-bit Windows machine.**

Prerequisites

Tool	Where to get it	Notes
Python 3.10+ (64-bit)	python.org	Must be 64-bit; add to PATH
Git for Windows	git-scm.com	Provides Git Bash for <code>scripts/build.sh</code>

Tool	Where to get it	Notes
PicoSDK 11.1.418	picotech.com/downloads	Reboot after install
Inno Setup 6	jrsoftware.org/isinfo.php	Per-user install to %LOCALAPPDATA% is fine

Install project dependencies (from Git Bash or cmd.exe):

```
pip install -e ".[dev]"
```

Running the build

```
# Run from the repo root
# Full pipeline: collect DLLs → PyInstaller → Inno Setup
./scripts/build.sh

# Individual steps
./scripts/build.sh dlls           # collect PicoScope DLLs into drivers/ only
./scripts/build.sh pyinstaller    # PyInstaller only (skips DLL collection)
./scripts/build.sh installer      # Inno Setup only (requires dist/ to exist)
./scripts/build.sh wheel          # Python wheel + sdist (runs anywhere, no Windows tooling)

# Flags, any position
./scripts/build.sh all clean       # force a full PyInstaller cache wipe
./scripts/build.sh all nodlls      # skip DLL collection -> driver-less bundle
```

`nodlls` is what CI uses; see [Automated releases](#). `wheel` is the only target that runs off Windows.

`python -m build` **does not work from the repo root**. This repo has its own `build/` directory, which shadows the `build` PyPI package as an implicit namespace package: `import build` succeeds and `python -m build` fails with `*No module named build.__main__*`. `./scripts/build.sh wheel` runs it from a scratch directory to sidestep this — use that rather than calling the frontend directly.

Output

Path	Description
<code>dist/rev80/rev80.exe</code>	Standalone executable (no install needed)
<code>installer/Output/Rev80Setup-<version>.exe</code>	Installer with Start Menu shortcut and uninstaller
<code>dist/rev80-<version>-py3-none-any.whl</code>	Python wheel (<code>./scripts/build.sh wheel</code>)
<code>dist/rev80-<version>.tar.gz</code>	Source distribution

Known constraints

- **64-bit only** — PicoSDK DLLs are 64-bit; 32-bit Python will not work.
- **DearPyGui pinned to 2.0.0** — versions above 2.0.0 have a known viewport crash on Windows.
- **USB kernel driver** — `ps4000a.dll` is the user-mode library; the USB kernel driver is installed separately by PicoSDK. Reboot required before first hardware connection.
- **Code signing** — the installer and executable are unsigned; Windows SmartScreen will warn on first run. Sign with `osslsigncode` and a certificate if you need this.

- **Local builds still need real Windows hardware** — PyInstaller cross-compilation isn't reliable and PicoSDK's DLLs are Windows-only, so a full local build means: boot into Windows, pull, run `scripts/build.sh`. Tagged releases are automated instead — see below.

Automated releases

Pushing a `vX.Y.Z` tag runs `.github/workflows/release.yml`, which builds the installer and the wheel and attaches both to a **draft** GitHub Release. Review it, then publish by hand — the exe is unsigned, so a release is worth a look before it goes out.

```
git tag v0.2.0
git push origin v0.2.0
# -> test gate -> wheel (ubuntu) + installer (windows) -> draft release
```

Four things about it are worth knowing before you rely on it:

- **CI installers are driver-less.** GitHub's hosted Windows runners have no PicoSDK and it has no reliable unattended install, so the release build passes `nodlls`. The bundle works, but the user must install PicoSDK themselves; the installer warns when it is missing. A local `./scripts/build.sh` is unchanged and still bundles `ps4000a.dll + picoipp.dll`.
- **A tag trigger ignores branches.** GitHub Actions has no notion of “tagged on main” — any `v*` tag, on any branch, builds a release. Deliberate, but it means a stray tag produces a draft.
- **The version comes from `git describe`.** Every job checks out with `fetch-depth: 0`; without tags `setuptools_scm` falls back to `0.0.0+unknown` and the release would ship `Rev80Setup-0.0.0+unknown.exe` with nothing failing. Both build jobs assert against that string, because it is the failure here that is otherwise silent.
- **The wheel is a release asset, not a PyPI package.** `picosdk` is a direct git URL dependency and PyPI rejects those. Publishing to PyPI would mean restructuring that dependency first.

Legacy `rc0.x` tags predate this and do not match `v*`; they will not trigger anything.

Rehearsing it

Use a throwaway tag rather than a real version the first time, and check that the installer job's log reports a real version rather than the fallback:

```
git tag v0.0.1-citest && git push origin v0.0.1-citest
# ...then delete the draft release and:
git push origin :v0.0.1-citest && git tag -d v0.0.1-citest
```